Section 5 — Agent Architectures

How to Structure and Organise Agents

Author: Suresh D R | AI Product Developer & Technology Mentor

---

Table of Contents

#	Topic
5.1	Single-Agent Systems
5.2	Multi-Agent Systems
5.3	Hierarchical Agents
5.4	Manager-Worker Pattern
5.5	Collaborative vs Competitive Agents
5.6	Advanced Patterns
5.7	When to Use Which Architecture
5.8	Section Summary

What This Section Is About

Section 4 taught how agents remember. Section 5 teaches how agents are **structured**.

When you build an agent system in the real world, one of the most important decisions you make is: how many agents do I need, how do they talk to each other, and who is responsible for what?

Get this wrong and your system is slow, unreliable, and hard to debug. Get it right and you have a system that is fast, accurate, and easy to extend.

By the end of this section you will understand six different ways to structure an agent system — and most importantly, you will know exactly when to use each one and why.

5.1 Single-Agent Systems

> **Analogy:** A general practitioner (GP) at a clinic. One doctor who handles everything — checks your temperature, listens to your heart, reads your blood reports, prescribes medicine, and gives you advice on diet. For most patients with common problems, a single GP is all you need. Fast, efficient, no coordination needed. But if you need heart surgery, you need a specialist.

What Is a Single-Agent System?

One agent with access to multiple tools. The agent does all the thinking and all the action itself — there is no delegation to another agent. The same LLM reads the goal, plans the steps, calls the tools, reads the results, and writes the final answer.

```
SINGLE-AGENT STRUCTURE:

  [You]
    |
    ▼
  [Agent — GPT-4o Brain]
    |
    ├──→ [Tool: web_search]
    ├──→ [Tool: calculator]
    ├──→ [Tool: knowledge_base]
    └──→ [Tool: send_email]
    |
    ▼
  [Result delivered to you]

One brain. Multiple tools. One loop. Done.
```

A Real Example — Logistics Route Agent (Single)

```
Task: "Find the fastest route from Pune to Mumbai for a delivery
       van, avoiding toll roads, and estimate the fuel cost."

Single agent workflow:
  Thought: I need route information. I will search for it.
  Action:  web_search("Pune to Mumbai route avoiding tolls")
  Thought: Got route options. Now I need the distance.
  Action:  knowledge_base("Pune Mumbai distance toll free")
  Thought: Distance is 180 km via Sinhagad Road. Now calculate fuel.
  Action:  calculator("180 / 12 * 95")  (180km, 12kmpl, Rs95/litre)
  Thought: Fuel cost is Rs 1,425. I have all the information needed.
  Final Answer: "Take Sinhagad Road via Katraj — 180 km, no tolls.
                 Estimated fuel cost: Rs 1,425 at current petrol prices."
```

Simple task. One agent. Handled completely. No need for multiple agents.

When One Agent Is Enough

Single-agent works perfectly when:

- The task can be completed in under 15-20 steps
- The task is in one domain (no deep specialisation needed)
- The tools available are sufficient for the job
- Quality checking is not critical
- Speed matters more than depth

When Single-Agent Breaks Down

```
Problem 1 — Too many steps:
  Task: "Research the top 50 logistics companies in India,
```

```
         analyse each one, and produce a detailed report"
  Problem: 50 companies × 3–4 steps each = 150–200 steps
           One agent running 200 steps loses context,
           makes mistakes, and the result is inconsistent.

Problem 2 — Needs deep specialisation:
  Task: "Write the code for a route optimisation algorithm,
         test it rigorously, and write technical documentation"
  Problem: Writing good code requires deep coding expertise.
           Writing good tests requires deep testing expertise.
           Writing good docs requires different skills entirely.
           One generalist brain does all three poorly.

Problem 3 — Sequential bottleneck:
  Task: "Research 10 delivery zones simultaneously and
         create plans for all of them"
  Problem: Single agent is sequential — zone 1, then zone 2, then zone 3.
           If zone 2 takes 5 minutes, you wait 5 minutes.
           10 zones = 50 minutes. Multi-agent = 5 minutes (parallel).

Problem 4 — No objective review:
  Task: "Write a report and make sure it is accurate"
  Problem: The same brain that wrote it is the same brain reviewing it.
           It is biased toward its own output. Errors get missed.
```

5.2 Multi-Agent Systems

> **Analogy:** A hospital. When you walk in with a complex condition, you do not see just one doctor. You see a team — a GP who assesses you, a specialist who examines the specific organ, a radiologist who reads the scans, a pharmacist who manages the medication. Each expert does one thing and does it better than any generalist could. Their combined work produces a better outcome than any single doctor alone.

What Is a Multi-Agent System?

Multiple AI agents working together to complete a task. Each agent is a specialist with its own role, tools, and responsibilities. Agents communicate with each other — one agent's output becomes another agent's input.

```
MULTI-AGENT STRUCTURE:

  [You]
    |
    ▼
  [Orchestrator Agent]  ← coordinates everything
    |
    ├──→ [Research Agent]  ← specialist: finds information
    |          |
    |          └──→ tools: web_search, read_url
    |
    ├──→ [Analysis Agent]  ← specialist: analyses data
    |          |
    |          └──→ tools: calculator, data_processor
    |
    ├──→ [Writing Agent]   ← specialist: writes the output
    |          |
    |          └──→ tools: document_writer
    |
    └──→ [Review Agent]    ← specialist: checks quality
               |
               └──→ (reads output from Writing Agent)
    |
    ▼
  [Result delivered to you]
```

A Real Example — Competitor Analysis (Multi-Agent)

```
Task: "Produce a complete competitor analysis report for our
       logistics company — compare Delhivery, Ekart, Shadowfax,
       and Blue Dart on pricing, delivery speed, and coverage."

ORCHESTRATOR AGENT creates the plan:
```

```
  "I will delegate research on each company to Research Agents,
   then send all data to the Analysis Agent, then to Writing Agent,
   then to Review Agent."

RESEARCH AGENT A (runs simultaneously):
  Researches Delhivery — pricing, speed, coverage, news
  Returns: structured data on Delhivery

RESEARCH AGENT B (runs simultaneously):
  Researches Ekart — pricing, speed, coverage, news
  Returns: structured data on Ekart

RESEARCH AGENT C (runs simultaneously):
  Researches Shadowfax — pricing, speed, coverage, news
  Returns: structured data on Shadowfax

RESEARCH AGENT D (runs simultaneously):
  Researches Blue Dart — pricing, speed, coverage, news
  Returns: structured data on Blue Dart

All four run AT THE SAME TIME → 4x faster than sequential

ANALYSIS AGENT receives all four data sets:
  Compares pricing across all four
  Identifies strengths and weaknesses of each
  Finds market positioning gaps
  Returns: analysis and findings

WRITING AGENT receives analysis:
  Writes the executive summary
  Writes the detailed comparison sections
  Formats the report professionally
  Returns: complete draft report

REVIEW AGENT receives the draft:
  Checks for factual accuracy
  Checks for missing information
  Checks for consistency
  Returns: reviewed report with corrections
```

```
FINAL RESULT: Complete competitor analysis in a fraction
              of the time a single agent would take
```

The key insight: the research agents run simultaneously (parallel), saving enormous time. The analysis, writing, and review run sequentially because each depends on the previous.

5.3 Hierarchical Agents

> **Analogy:** A construction project. The project manager never picks up a brick — they plan, delegate, and coordinate. The site foreman manages a team of workers. The workers build. Three levels: planner, coordinator, executor. Each level has a clear role. The planner should not be laying bricks. The workers should not be designing the building.

What Is a Hierarchical Agent System?

A structured system with at least two levels:

Level 1 — High-Level Planner:
Receives the overall goal. Breaks it into sub-goals. Delegates each sub-goal to a specialist agent. Never executes tools directly. Only plans and coordinates.

Level 2 — Execution Agents:
Receive one specific sub-goal. Execute it using tools. Never plan the overall strategy. Just do their assigned task and report back.

```
HIERARCHICAL STRUCTURE:

  LEVEL 1: PLANNER AGENT
    Receives: "Create a full logistics optimisation plan for 5 cities"
    Breaks into:
      Sub-goal 1: Analyse Mumbai routes        → delegates to Agent A
      Sub-goal 2: Analyse Pune routes          → delegates to Agent B
      Sub-goal 3: Analyse Thane routes         → delegates to Agent C
      Sub-goal 4: Combine and prioritise       → delegates to Agent D
      Sub-goal 5: Write final recommendations  → delegates to Agent E
```

```
  LEVEL 2: EXECUTION AGENTS
    Agent A: Does only sub-goal 1 (Mumbai routes)
    Agent B: Does only sub-goal 2 (Pune routes)
    Agent C: Does only sub-goal 3 (Thane routes)
    Agent D: Does only sub-goal 4 (combine results)
    Agent E: Does only sub-goal 5 (write recommendations)
```

Why Hierarchy Helps

```
WITHOUT hierarchy (single agent trying to do everything):
  Agent jumps between high-level strategy and low-level execution
  Gets confused about what level it is operating at
  Mixes planning and doing — neither is done well
  Hard to debug — you cannot tell if the problem is the plan or the execution

WITH hierarchy:
  Planner focuses only on planning — it is better at it
  Executors focus only on execution — they are better at it
  If a result is wrong, you know exactly which level failed
  You can swap out any execution agent without changing the planner
  Easy to scale — add more execution agents for more sub-goals
```

Task Decomposition — How the Planner Breaks Down Goals

Good task decomposition is the most important skill in hierarchical agents. A bad decomposition leads to overlapping work, missing steps, and confused agents.

```
GOOD decomposition (clear, non-overlapping, complete):

  Goal: "Plan delivery operations for Mumbai next month"

  Sub-goal 1: Collect current data (agent reads reports, databases)
  Sub-goal 2: Identify bottlenecks (agent analyses the data)
  Sub-goal 3: Generate solution options (agent brainstorms approaches)
```

```
  Sub-goal 4: Evaluate each option (agent scores options against criteria)
  Sub-goal 5: Write the final plan (agent formats and presents recommendation)

  Each sub-goal is clear. No overlap. Every step is covered.

BAD decomposition (vague, overlapping, missing steps):

  Sub-goal 1: Look at delivery stuff
  Sub-goal 2: Come up with ideas
  Sub-goal 3: Write something

  "Delivery stuff" is vague — what exactly?
  "Ideas" overlaps with "write something"
  Steps to evaluate and refine are missing entirely
```

5.4 Manager-Worker Pattern

> **Analogy:** A restaurant kitchen. The head chef (manager) receives orders from the waiters, decides which dishes to prepare, assigns the salad to the salad station, the main course to the grill station, and the dessert to the pastry station. Each station (worker) makes one thing and passes it back. The head chef assembles the final plate. No station needs to know what the other stations are doing.

What Is the Manager-Worker Pattern?

A specific type of hierarchical system where one manager agent actively coordinates multiple worker agents in real time.

The Manager Agent:

- Receives the overall task
- Decides how to break it down
- Assigns each piece to the right worker
- Tracks which workers have finished
- Handles failures (reassigns if a worker fails)
- Assembles all worker outputs into the final result

Worker Agents:

- Each has a specific specialty and tools
- Receives one clearly defined assignment
- Completes it and returns the result
- Does not know or care what other workers are doing

Task Allocation Strategies

The manager has several ways to decide which worker gets which task:

```
Strategy 1 — Skill-based allocation:
  Match the task to the worker whose skills fit best.
  Writing task → Writing Agent
  Maths task   → Analysis Agent
  Search task  → Research Agent
  Best for: tasks with clearly distinct requirements

Strategy 2 — Load-based allocation:
  Assign to the least busy worker.
  If Research Agent A has 3 tasks queued,
  send the next task to Research Agent B (0 tasks queued).
  Best for: high-volume systems where throughput matters

Strategy 3 — Round-robin:
  Distribute evenly across all workers regardless of load or skill.
  Task 1 → Worker A, Task 2 → Worker B, Task 3 → Worker C,
  Task 4 → Worker A again.
  Best for: tasks that are all similar in nature

Strategy 4 — Priority-based:
  Urgent tasks go to the fastest or most capable worker.
  Non-urgent tasks go to any available worker.
  Best for: systems with mixed priority levels
```

Handling Worker Failures

A good manager handles failures gracefully:

```
Normal flow:
  Manager assigns zone 5 research to Worker B
  Worker B completes it and returns the result
  Manager continues assembling

Failure scenario:
  Manager assigns zone 5 research to Worker B
  Worker B fails — tool error, timeout, or bad output

  Manager options:
    1. Retry: reassign the same task to Worker B again
    2. Reassign: send the task to Worker C instead
    3. Simplify: break the task into smaller pieces and retry
    4. Skip + flag: skip zone 5, note it in the output, continue

Good managers try options 1, 2, 3 before resorting to option 4.
```

5.5 Collaborative vs Competitive Agents

> **Analogy for Collaborative:** A relay race team. Each runner completes their leg and passes the baton to the next. Different strengths contribute to one shared goal.

> **Analogy for Competitive:** Two independent auditors reviewing the same financial statements. If they both reach the same conclusion, you are confident in it. If they disagree, you investigate further before making any decision.

Collaborative Agents — Working Together

Agents work together toward a shared goal. Each contributes their specialty. One agent's output is the next agent's input. The goal is achieved by combining their individual contributions.

```
Example — Delivery Report Generation:

  DATA AGENT collects yesterday's delivery statistics
      |
      ▼ passes data
  ANALYSIS AGENT finds patterns, calculates KPIs
      |
      ▼ passes analysis
  WRITING AGENT writes the report with insights
      |
      ▼ passes draft
  FORMATTING AGENT adds charts, tables, company branding
      |
      ▼
  Final report delivered to management

Each agent made the output better. The final result
is far better than any single agent could produce.
```

Competitive Agents — Checking Each Other

Two or more agents work on the same problem independently. Their outputs are compared. The most consistent answer wins. This dramatically reduces errors.

```
Example — Delivery Cost Estimate:

  AGENT A estimates cost independently:
    "Based on distance 180km, fuel at Rs 95/litre, 12kmpl —
     fuel cost = Rs 1,425. Add driver Rs 800. Total: Rs 2,225."

  AGENT B estimates cost independently:
    "180km route, petrol Rs 95/litre, efficiency 12kmpl —
     fuel = Rs 1,425. Driver cost Rs 800. Total estimate: Rs 2,225."

  Both agree: Rs 2,225 → High confidence → Use this figure

  Another scenario:
```

```
  AGENT A says: Rs 2,225
  AGENT B says: Rs 1,900  (forgot to include driver cost)

  They disagree → Trigger AGENT C (tiebreaker) to check
  AGENT C finds AGENT B missed driver cost
  Correct answer: Rs 2,225

Competitive agents catch errors that a single agent would miss.
```

Debate and Consensus

A more sophisticated form of competitive agents where agents argue with each other:

```
AGENT A proposes: "Use SV Road for the Mumbai to Pune route"
AGENT B challenges: "SV Road has construction until March.
                     NH-48 is actually faster right now."
AGENT A responds: "You are right about the construction.
                   NH-48 is the better option currently."
CONSENSUS: Use NH-48 for the Mumbai-Pune route

The debate produced a better answer than either agent
would have reached alone. AGENT A's initial suggestion was
overturned by AGENT B's current knowledge.
```

5.6 Advanced Patterns

Swarm Intelligence — Many Simple Agents

> **Analogy:** Ants building a colony. Each individual ant follows three simple rules — follow pheromone trails, deposit pheromones when finding food, avoid crowded paths. No ant has a blueprint of the colony. No ant is directing others. Yet collectively they build complex, efficient structures. This is swarm intelligence.

In agent systems, swarm intelligence means using many simple agents, each following simple rules, to collectively produce complex intelligent behaviour.

```
Example — Processing 500 customer complaints:

  WITHOUT swarm (single agent):
    Agent reads complaint 1 → categorises → responds
    Agent reads complaint 2 → categorises → responds
    ...
    500 complaints × 2 minutes each = 1000 minutes = 16+ hours

  WITH swarm (50 simple agents):
    Agent 1 handles complaints 1–10
    Agent 2 handles complaints 11–20
    Agent 3 handles complaints 21–30
    ...all 50 run simultaneously...
    500 complaints ÷ 50 agents × 2 minutes = 20 minutes

  Same result. 50x faster.
  Each agent is simple — it just categorises and responds.
  The swarm collectively handles the full load.
```

Key characteristics of swarm agents:

- Each agent is simple — it does one narrow task
- Agents run in parallel — no waiting for each other
- No central controller — agents are independent
- The complexity emerges from the collective, not the individual

---

Self-Improving Agents — Reflexion at Scale

An agent system where agents evaluate their own performance after each run and update their approach for the next run. This is Reflexion (from Section 2) applied at the system level.

```
Run 1: Agent attempts a task → completes it
       System evaluates the result (automatically or with human rating)
       Agent writes a reflection: "I forgot to check for empty inputs"
```

```
Run 2: Agent reads the reflection before starting
       Agent applies the lesson → better result

Run 3: Agent reads reflections from runs 1 and 2
       Applies both lessons → even better result

After 10 runs:
  Agent has accumulated 10 lessons
  It consistently avoids all previously-made mistakes
  Its performance improves measurably with every run
```

Agent Personas — Role-Based Design

Giving each agent a specific role, personality, and set of responsibilities makes multi-agent systems more predictable and easier to reason about.

```
Instead of:
  "Agent 1" and "Agent 2" and "Agent 3"

Use:
  SENIOR ANALYST AGENT
    Role: Provide expert data analysis
    Personality: Precise, data-driven, cautious about uncertainty
    Responsibility: Never present uncertain data as fact
    Tools: database queries, statistical functions

  CREATIVE STRATEGIST AGENT
    Role: Generate new ideas and approaches
    Personality: Bold, unconventional, explores alternatives
    Responsibility: Propose at least 3 different options for every problem
    Tools: web_search, brainstorming prompts

  DEVIL'S ADVOCATE AGENT
    Role: Challenge every conclusion
    Personality: Skeptical, rigorous, hard to convince
    Responsibility: Find the weakness in every proposal
    Tools: fact-checking, counter-argument generation
```

With clear personas, you know what to expect from each agent. The system is predictable. Debugging is easier — if the analysis is wrong you look at the Senior Analyst Agent. If no alternatives were proposed you look at the Creative Strategist.

5.7 When to Use Which Architecture

This is the most practical section. Here is a clear guide for every situation.

The Decision Guide

```
START HERE: How complex is the task?

Simple (1–5 steps, one domain, single user)
  → Use Single-Agent
  → Example: "What is the delivery cost from Pune to Mumbai?"

Medium (5–15 steps, clear sequence, one domain)
  → Use Single-Agent with good tools
  → Example: "Research 3 delivery routes and recommend the best"

Complex (15+ steps, multiple domains, quality matters)
  → Use Multi-Agent
  → Continue to next question...

_____________________________________________________

For Multi-Agent — what kind of complexity?

Tasks can run simultaneously?
  → Yes → Use Parallel Multi-Agent (multiple research agents)
  → No  → Use Sequential Multi-Agent (each depends on previous)

Need strict quality control?
  → Use Competitive Agents (two agents, compare outputs)
  → Or add a dedicated Review Agent after the Writing Agent
```

```
Task needs deep planning before execution?
  → Use Hierarchical Agents (Planner → Executors)
  → Example: Complex logistics planning across 12 cities

One agent coordinating many workers?
  → Use Manager-Worker Pattern
  → Example: Manager assigns zones to delivery route agents

High volume, same task repeated thousands of times?
  → Use Swarm Intelligence (many parallel simple agents)
  → Example: 5000 customer emails categorised simultaneously

Agent needs to improve over time with experience?
  → Add Self-Improving Pattern (Reflexion + episodic memory)
  → Combine with any architecture above
```

The Simple Truth — When to Use Each

Architecture	Use When	Do NOT Use When
Single-Agent	Task is simple, one domain, under 15 steps	Task is complex, needs specialisation
Multi-Agent Parallel	Multiple independent subtasks	Tasks that must be sequential
Multi-Agent Sequential	Tasks that build on each other	Tasks are independent (use parallel)
Hierarchical	Complex goal needs decomposition	Simple task — overhead not worth it
Manager-Worker	High volume of similar subtasks	One-off tasks
Collaborative	Different specialisations needed	One domain, one agent is enough
Competitive	Accuracy is critical, errors are costly	Speed is more important than accuracy
Swarm	Same task, huge volume, needs speed	Tasks that are all different
Self-Improving	Repeated tasks over long period	One-off or rarely repeated tasks

---

Real Examples Mapped to Architecture

Scenario	Architecture	Why
"What is the fastest route from Pune to Mumbai?"	Single-Agent	Simple, one step, one domain
"Research 5 delivery companies and compare them"	Multi-Agent Parallel	5 independent research tasks
"Write and review a logistics strategy document"	Collaborative (Writer + Reviewer)	Writer and Reviewer are separate specialists
"Verify this delivery cost calculation is correct"	Competitive (two agents)	Accuracy critical, need second opinion
"Plan entire logistics operations for 12 cities"	Hierarchical	Complex goal needs decomposition
"Process 5000 customer complaint emails today"	Swarm	Same task, massive volume, parallelism needed
"Our customer support agent keeps making the same mistake"	Self-Improving	Repeated task, learning needed over time
"Assign route planning tasks to 10 specialist zone agents"	Manager-Worker	One coordinator, many specialists

---

The Cost of Getting It Wrong

```
Using single-agent when you need multi-agent:
  Output is low quality — generalist cannot match specialist depth
  Takes much longer — sequential when parallel was possible
  Hard to debug — everything is one black box

Using multi-agent when single-agent was enough:
  Unnecessary complexity
  More points of failure
  Slower (coordination overhead)
  Harder to maintain

The right architecture for the right task is one of the
most important decisions in building production agent systems.
Getting it right saves weeks of debugging and rebuilding.
```

5.8 Section Summary

Architecture	Structure	Best For	Key Advantage
Single-Agent	1 brain + multiple tools	Simple focused tasks	Simple, fast, easy to debug
Multi-Agent Parallel	Multiple agents running at same time	Independent subtasks	Speed — 4x, 10x, 50x faster
Multi-Agent Sequential	Agents in a chain	Tasks that build on each other	Specialisation at each step
Hierarchical	Planner + Executors	Complex goals needing decomposition	Clear separation of planning and doing
Manager-Worker	1 manager + N workers	High volume of similar subtasks	Coordination with specialisation
Collaborative	Specialists in sequence	Different expertise needed at each step	Each agent contributes their best
Competitive	2+ agents, same task	Accuracy-critical decisions	Errors caught before they reach you
Swarm	N simple parallel agents	Massive volume, same task	Throughput — scale horizontally
Self-Improving	Any agent + reflection loop	Repeated tasks over time	Gets better automatically

---

The One Rule to Remember

Start simple. Add complexity only when the simple version breaks.

```
Step 1: Try single-agent. Does it produce good results?
  Yes → You are done. Ship it.
  No  → Move to Step 2.

Step 2: What is breaking?
  Quality too low → Add specialised agents
  Too slow        → Add parallel agents
  Making mistakes → Add competitive/review agents
  Too complex     → Add hierarchy
  Too much volume → Add swarm
  Repeating mistakes → Add self-improvement

Every architecture in this section exists because
```

```
the simpler architecture broke for a specific reason.
Know the reason before choosing the architecture.
```

Suresh D R — AI Product Developer & Technology Mentor
Agentic AI — Section 5 | Generative AI Training Course